rosetta-flow

A proof of concept: an open, on-premise auditor for AI-agent runs, built with Apertus.

TEAM
Anes Mulalic (solo)

DEMO
youtu.be/zZuEkJsSdbQ

CODE
github.com/anesask/rosetta-flow

WEIGHTS
anesdevtoday/rosetta-flow-auditor-lora

DATASET
anesdevtoday/rosetta-flow-traces

1 Summary

Companies now let AI agents issue refunds, answer customers and release production lots. Every run leaves a trace (messages, tool calls, tool results), but nobody can read thousands of them, and banks, insurers, manufacturers and public offices cannot send them to a foreign cloud tool. **rosetta-flow turns Apertus into an on-prem auditor for AI agents: the small model (8B) screens every run, the large one (70B) confirms only the flagged ones, and every finding comes with verified evidence.** The auditor investigates a run like a code repository (one file per step, read and search tools, a typed report), and code checks every quoted piece of evidence against the step it cites, so a reviewer sees the exact step and a quote that is really there, never an invented one.

A proof of concept, built in ten days by one person. It shows that the pieces work together end to end on open Swiss models: investigation with tools, verified evidence, two tiers, offline operation and human review. It does not yet show production accuracy: training the small model was limited by budget and compute (section 3), the data is simulated, and on an agent from another domain the auditor needs adaptation (section 5.5).

What we measured, on 262 held-out runs in two domains (refunds, manufacturing quality) and four languages:

- **Half the false alarms of the 70B alone** (0.19 vs 0.38) at about the same balanced accuracy (0.68 vs 0.70), with the 70B reading only about 56 % of the runs. **The price is recall:** the two tiers flag 71 of the 131 broken runs, the 70B alone 101, because the 8B clears some broken runs that the 70B then never sees.
- **Scale on one GPU:** the 8B screens about 1,500 to 1,900 runs an hour on a single A100, roughly 40,000 a day; at CSCS prices an audit costs under a tenth of a Rappen.
- **Runs offline:** audits with the network cut add no outside traffic, measured with the kernel's byte counters.
- **Built for oversight:** human decisions on findings go into a tamper-evident trail, the kind of record that FINMA's AI guidance and the EU AI Act ask for.

A live demo lets anyone sabotage an agent run and watch the two tiers catch it. We report what did not work just as plainly (sections 5.3 to 5.5): fine-tuning the 8B made it more reliable but not more accurate, a fast screener trained with reinforcement learning gave no gain, and on an agent from another domain (tau-bench) the auditor needs adaptation.

Who it is for. Organisations that run agents on regulated work and must keep data inside: banks and insurers (FINMA Guidance 08/2024), manufacturers whose production networks are often air-gapped and whose quality systems need audit trails (EU GMP Annex 11), and public administration. The two domains we built are templates: a refunds policy stands in for any customer-facing rulebook; the manufacturing domain mirrors a real quality process and was designed by an author who works in manufacturing.

2 Architecture

The system has a build-time half (online, once) and a runtime half (inside the customer network); details in `docs/architecture.md`.

Trace format. Our own small JSON (spans of kind `user_message`, `agent_message`, `tool_call`, `tool_result`), one object per run. It maps onto OpenTelemetry `gen_ai.*` spans, but we have not built that importer yet (tau-bench runs come in through their own converter). Labels live in a separate file the auditor never sees.

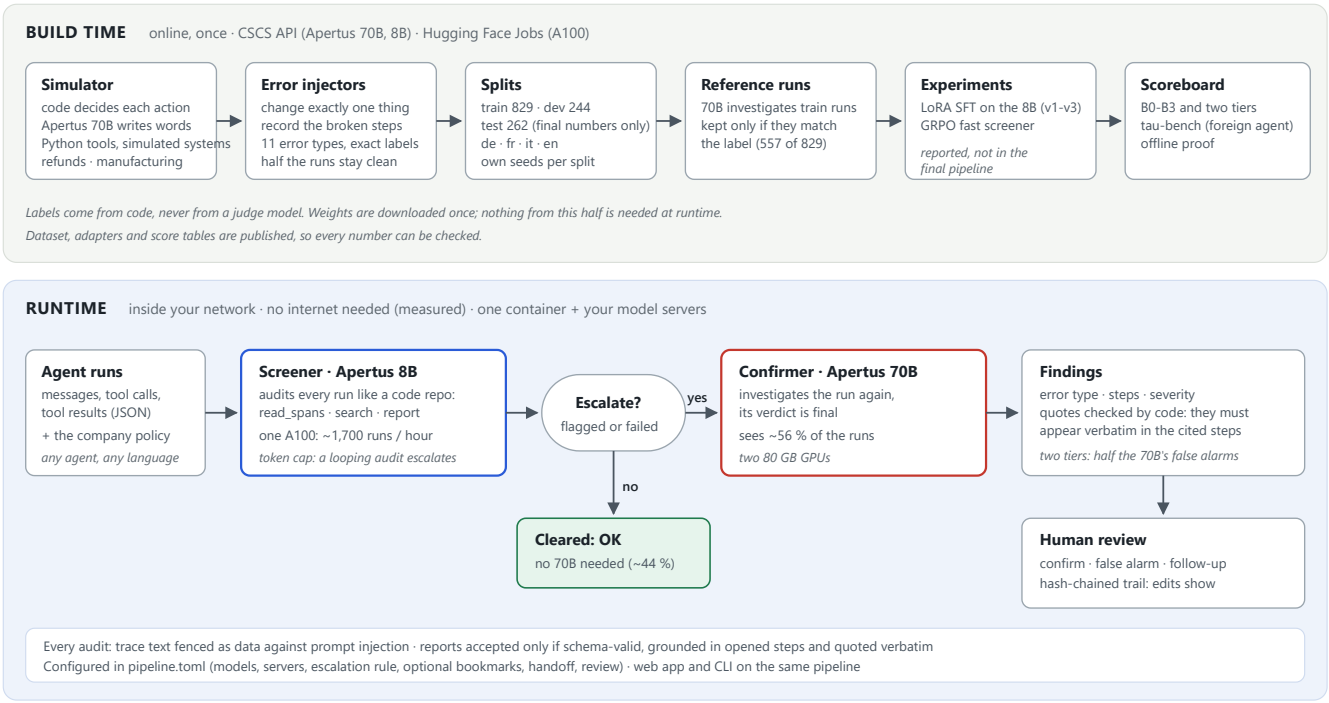


Figure 1. Build time (online, once) and runtime (inside the customer network).

Investigation harness (`src/rosetta_flow/harness/`). The trace becomes a virtual repository: the task message holds the company policy and a one-line index per step; the model calls `read_spans` , `search` and `list_spans` and finishes with `report` , a typed JSON object (error type from a taxonomy of 11, span ids, verbatim evidence, explanation, expected behaviour, severity, confidence). A report is accepted only if it (1) passes the schema, (2) cites only steps the model actually opened, and (3) quotes evidence that appears verbatim in those steps. All trace content is fenced as data with a random per-audit tag (prompt-injection defence). Robustness fixes found from transcripts: bounded repair of almost-valid JSON, cleaning template tokens that leak into tool names, keeping the last schema-valid report as a marked fallback at the call limit (16), and an optional per-audit token cap so a screener stuck in a correction loop escalates fast.

Two-tier pipeline (`src/rosetta_flow/pipeline.py` , configured in `pipeline.toml`). The screener (Apertus 8B) audits every run. An escalation rule sends runs it flags, or fails on, to the confirmer (Apertus 70B), whose verdict is final; runs the screener clears stay cleared. Each tier can point at its own server. Optional parts: *bookmarks* (the screener marks suspicious steps), *handoff* (the confirmer receives them as fenced pointers, not instructions) and *human review* (`review.py` : confirmed / false alarm / follow-up, appended to a log in which every entry carries the hash of the previous one, so later edits are detectable).

Single-pass screener (experiment) (`screen.py` , `train/rl.py`): a fast mode that reads the whole run once and answers `{"verdict", "p_issue", "suspicious_spans"}` , trained with GRPO on rewards from our exact labels. It did not work (section 5.4) and is not part of the final pipeline.

Web app (`src/rosetta_flow/app/` ; front end in `web/` , React and Chakra UI built into static files that the container serves, so `make run` needs no Node). *Break the agent*: pick one of 16 recorded clean runs (two domains, four languages), optionally sabotage it with one of our error injectors, and watch the two tiers audit it live as a node flow: each tier's tool calls stream in as they happen, then the verdict, the evidence check and this run's cost against the 70B alone. An investigation view replays the audit step by step (each tool call explained, the steps it read, the decision and what two tiers save), and a transcript view shows the whole run with every tool, what it does and its full answer. *Fleet* replays the 262 measured test runs with a Sankey of what happened to every run and the cost of both set-ups. *Review* lets a person confirm or reject each 70B finding in the hash-chained trail. Trace text is always rendered as text, never as HTML.

Target architecture (mandatory)

rosetta-flow is deployable **(a) on-premise** and **(b) air-gapped**. At runtime nothing leaves the network: one container (harness, evidence check, web app) talks to two local vLLM servers through the OpenAI-compatible API, the 8B on one GPU with at least 24 GB and the 70B on two 80 GB GPUs. Only `LLM_BASE_URL` changes between modes. For judges, `make run` points the same container at the CSCS API instead, so no GPU is needed.

Offline run, measured (`scripts/hf_offline.sh` , evidence in `results/offline-proof/`). One GPU job in three phases: online, download the 8B weights and copy code and demo runs to local disk; offline, serve the 8B from local files with every Python process behind an egress guard (any connection off the machine refused and logged) and audit "Break the agent" cases; online again, upload the evidence. The container allows no firewall, so the kernel's byte counters decide: during the audits the interface carried 31 B/s out

and 482 B/s in, the same as an idle minute before (32 / 545 B/s, platform background), while 298 KB went over loopback between auditor and model. An attempt to reach huggingface.co from the auditor was refused. On the 8B alone, the control run was cleared and the unapproved lot release caught on the broken steps.

Dependency	Build time	Runtime (on-prem / air-gapped)
CSCS API (Apertus 70B, 8B)	trace simulation, reference investigations, B3 evaluation	none (judges' demo only)
Hugging Face Hub	model download; publish dataset and adapters	none: weights copied to local disk once
Hugging Face Jobs (L4, A100)	LoRA training, vLLM evaluation of the 8B systems	none
vLLM, Docker, Python 3.13	evaluation, development	model servers, the container
Python packages	`openai`, `pydantic`, `regex`, `json-repair`, `fastapi` (pinned in `requirements.txt`)	same, inside the container

3 Use of Apertus

- **Models:** `swiss-ai/Apertus-v1.5-70B` (CSCS API); Apertus v1.5 8B as the text-only conversion `andreasmartin/apertus-v1.5-8b-text` (FHNW) for fine-tuning and local serving, `swiss-ai/Apertus-v1.5-8B` via the CSCS API. The organisers confirmed on Discord that the conversion counts as Apertus.
- **How it is used:** agents / tool use (both tiers investigate with native tool calls), evaluation (the 70B as reference auditor), synthetic data generation (the 70B plays customers and writes agent replies), fine-tuning (LoRA SFT and GRPO on the 8B).
- **Where it runs:** CSCS API at build time; vLLM v0.31 with LoRA adapters on Hugging Face Jobs for evaluation; local weights at runtime.
- **No other models** are part of the system or the evaluation; labels come from code, not from a judge model.

Practical notes for rebuilding: Apertus produces activations of about 118,000 from layer 3 on, which overflows fp16 and gave NaN losses on T4 GPUs; training therefore runs in bf16 on L4 or A100. The chat template accepts only string message content, and the 8B occasionally leaks template tokens into tool names.

Fine-tuning (an honest experiment). The 70B audited 829 training runs; 557 investigations matched their label (rejection sampling, up to 3 attempts) and 414 balanced examples became SFT data. Three LoRA adapters (rank 16, all linear layers): v1 4-bit QLoRA, 2 epochs (L4, 92 min, $1.23); v2 bf16, 3 epochs (A100, 38 min, $1.61); v3 as v2 but keeping the 70B's rule-by-rule analysis before each report (47 min, $1.97). The adapters are published with a model card on Hugging Face.

What the budget allowed. Free Kaggle T4 GPUs could not train Apertus (no bf16 hardware, fp16 overflows, emulated bf16 took over 7 minutes per step), so every run was paid from a small personal Hugging Face credit at USD 1 to 4 each, and the CSCS API was capped at 130M tokens after the organisers' fair-use note, which limited how many 70B investigations we could generate. The result: 414 training examples, one run per configuration, one seed, no search over settings, 80 GRPO steps, and the 8B evaluated at 16k context. Sections 5.3 and 5.4 are results under this budget, not proof that training cannot help; section 8 says what we would do with more.

4 Data

All data is **synthetic and generated for this project**; it contains no personal data. Two domains with written policies: *refunds and customer support* (Example Shop AG, 8 tools, policy `refunds-v1`) and *manufacturing quality* (Example Präzision AG, 15 tools over five systems: drawings, MES, ERP, QMS, gauge calibration; policy `mfg-v1`). The companies and people are fictional; the processes and rules are modelled on real ones. The manufacturing domain follows a typical quality process in precision manufacturing (released drawing revisions, calibrated gauges, non-conformance reports with a hold on the lot, approval before release, warm-up and first-piece checks after a stop), from the concept of an earlier, unsubmitted project by the author (NeuralLine; no code reused). The refunds domain follows common online-shop return terms. Simulated data lets every label be exact: we know which step is broken because code broke it.

Generation. Code decides what the agent does (a scripted agent over a seeded, simulated company database), Apertus 70B writes the customer's messages and the agent's words, and real Python tools answer, so tool results are always consistent. A verbaliser rejects drafts that invent names, handovers or facts, so clean runs stay clean. Error injectors then change exactly one thing (flip a tool result, remove a required check, change an amount, translate a reply, leak a masked IBAN, release a lot without approval) and record which steps are wrong; about half of all runs stay clean.

Category	Error types (11, each with an exact label)	What goes wrong
Grounding	`contradicts_tool_result`, `hallucinated_fact`, `ignored_tool_error`	the reply says something the tools did not return
Process	`skipped_required_step`, `wrong_order`, `false_completion`	a required check is missing, late, or claimed but not done
Tool use	`wrong_tool_args`, `wrong_tool`	the right action with wrong inputs, or the wrong system
Policy	`policy_violation`, `data_leak`	a rule of the company policy is broken, data of another customer is exposed
Communication	`language_mismatch`	the reply is in a different language than the customer's

The runs are split by seed; the test set was not looked at until the final runs.

Set	Refunds	Manufacturing	Seeds (simulation / injection)	Use
train	409	420	101 / 11	reference investigations, LoRA, GRPO
dev	120	124	41 / 7	all harness and training decisions
test	130	132	202 / 13	final numbers only

Languages: German, French, Italian and English. Generation cost at the CSCS API: about 116M tokens, CHF 7.2 in total (including all development runs). The dataset (traces, labels, model outputs, manifests) is published on Hugging Face under CDLA-Permissive-2.0 (tau-bench subset MIT).

5 Evaluation

Task. Given a trace and the policy, decide whether the agent made an error, and if so name the type, cite the steps and quote evidence. **Baselines** (each row changes one thing): B0 the 8B with the whole trace in one prompt; B1 the 8B with the harness; B2 the 8B with harness and LoRA v2; B3 the 70B with the harness. Decisions were made on dev; the test set was used for the final runs below.

Metrics, per domain. A run is *positive* if it holds an injected error and *flagged* if the auditor reports an issue; a failed audit counts as wrong on every run (a false alarm on a clean run, a miss on a broken one). A *hit* is a flagged broken run with a credible finding: it cites at most 4 steps, at least one of them broken, and at least one of its quotes appears verbatim in a cited step.

Metric	Definition	What it shows
Balanced accuracy	$\mathrm{BA} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right)$	0.5 is a coin toss; flagging everything also scores 0.5, while detection F1 would give it 0.67
False alarms	$\mathrm{FA} = \frac{FP}{FP+TN}$	share of clean runs a reviewer would open for nothing
Strict F1	$F_1 = \frac{2PR}{P+R}$ $P = \frac{H}{H+FP_s}, R = \frac{H}{H+FN_s}$	H hits; FP_s flags on clean runs or on the wrong step; FN_s broken runs without a hit
Evidence verified	$\mathrm{EV} = \frac{F_v}{F}$	F findings, F_v of them with every quote verbatim in a cited step: invented evidence, caught by code
Cost per run	$c = n \cdot p_m$	n tokens (input and output), p_m CSCS price of model m: 70B CHF 0.092/M, 8B CHF 0.012/M
Two-tier cost	$c_2 = c_{8B} + e \cdot c_{70B}$	e share of runs escalated to the 70B (0.55 to 0.57 on test)
Brier score	$B = \frac{1}{N}\sum_i (p_i - y_i)^2$	calibration of a probability p_i against the label $y_i \in \{0, 1\}$; always answering 0.5 gives 0.25

5.1 Held-out test set (262 runs)

Setup	Balanced acc. (refunds / mfg)	Strict F1	False alarms	Evidence verified	Failed audits	CHF per 1,000 runs
B0: 8B, one prompt	0.50 / 0.52	0.20 / 0.20	0.98 / 0.89	0.25 / 0.21	1 / 4	0.03
B1: 8B + harness	0.65 / 0.50	0.44 / 0.30	0.39 / 0.52	0.67 / 0.45	5 / 11	0.14 / 0.20
B2: 8B + harness + LoRA	0.62 / 0.42	0.47 / 0.23	0.49 / 0.56	0.72 / 0.77	0 / 1	0.11 / 0.14
B3: 70B + harness	0.69 / 0.71	0.60 / 0.52	0.48 / 0.27	0.85 / 0.83	0 / 0	1.04 / 1.14
Ours: B1 screens, B3 confirms	**0.71 / 0.64**	0.59 / 0.43	**0.21 / 0.17**	0.86 / 0.88	0 / 0	0.76 / 0.90

Costs as defined above. In the two-tier system the 70B saw 57 % (refunds) and 55 % (manufacturing) of the runs. A second, independent measurement of the two tiers, before the last harness fixes, gave the same picture (0.69 / 0.67, false alarms 0.25 / 0.09).

On own hardware. Measured: the 8B audited the 130 refunds test runs in 251 s and the 132 manufacturing runs in 313 s on one A100 (vLLM, 16 audits in parallel), 1.9 and 2.4 GPU-seconds per run, so **one GPU screens about 1,500 to 1,900 runs an hour, roughly 40,000 a day**. Estimated, not measured (we served the 70B only through the API): scaling GPU time by the parameter ratio gives about 15 to 17 GPU-seconds per 70B audit, so the two tiers need about 30 % less GPU time than the 70B alone. Two tiers are mainly a quality gain (half the false alarms) at a lower cost; their practical advantage on premises is that a single 24 GB GPU covers every run and the expensive 70B servers only see what is flagged.

5.2 What the numbers say

- **The harness is what makes the 8B usable.** In one prompt the 8B flags almost every run and only a quarter of its quotes exist (B0). With the investigation harness it becomes a real auditor (B1).
- **Two tiers trade a little recall for far fewer false alarms.** Runs the 8B clears are rarely wrong in refunds, so the 70B's verdict on the flagged ones removes most false alarms (0.48 to 0.21). In manufacturing the 8B lets more errors through as "ok" (two-tier recall 0.44 vs 0.70 for the 70B alone): the main limitation. Fewer false alarms is the practical gain: it halves the runs a person has to review.
- **Investigating beats reading everything, for the 70B too.** A cheaper confirmer that checks the screener's findings with the whole run in one prompt (`mode = "verify"`, no tools) did worse on dev than the investigating 70B: balanced accuracy 0.64 / 0.65 vs 0.73 / 0.69, false alarms 0.27 / 0.19 vs 0.18 / 0.11, at a fifth of the confirmer's tokens. It stays an option, not the default.

5.3 Fine-tuning: reliability up, accuracy flat

Dev set (244 runs)	Balanced acc. (refunds / mfg)	Recall	False alarms	Failed audits
B1: 8B + harness	0.66 / 0.50	0.63 / 0.35	0.32 / 0.35	20
v2: LoRA, bf16, 3 epochs	0.60 / 0.48	0.72 / 0.42	0.52 / 0.47	3
v3: v2 + 70B's analysis	0.56 / 0.48	0.15 / 0.05	0.03 / 0.08	2

Without the 70B's reasoning (only its tool calls) the 8B learned to flag too eagerly; with it, too cautiously. LoRA cut failed audits by about 90 % and made audits 2 to 3 times faster, but did not beat the untrained 8B on accuracy. Its stated confidence was uncalibrated (almost always >= 0.95), so a threshold could not help either. Following the kill rule we set in advance, fine-tuning is reported as an experiment and the final screener is the untrained 8B with the harness.

5.4 Single-pass screener with GRPO: no gain

The idea: a fast, calibrated first pass whose probability sets the escalation threshold, so cost and recall become a dial. GRPO with LoRA on the 8B (80 steps, 4 prompts x 8 answers, A100, 85 min, $3.56), evaluated on dev against the same model untrained: balanced accuracy 0.53 / 0.48 vs 0.54 / 0.48, false alarms 0.83 / 0.68 vs 0.87 / 0.76, Brier score 0.37 / 0.40 vs 0.37 / 0.41 (worse than always answering 0.5, which gives 0.25), only 5 to 7 distinct probability values. Reading a whole run in one pass, the 8B does not separate faulty from clean runs (like B0), and GRPO did not change that. The weakness is single-pass reading, not the amount of training, so we did not run a second round.

5.5 Runs from an agent we did not write (tau-bench)

100 recorded airline runs from tau-bench (MIT, (c) 2024 Sierra), 50 with a solved and 50 with a failed task, converted to our format with the airline's policy. tau-bench says only whether the final database state matched the customer's hidden goal, not which step went wrong, so we score detection only.

tau-bench airline (100 runs)	Balanced acc.	Failed tasks flagged	Solved tasks flagged	Failed audits
70B + harness	0.42	0.44	0.60	3
8B + harness	0.30	0.24	0.64	41 (16k context too small)
8B screens, 70B confirms	0.44	0.34	0.46	3

All systems are below chance at predicting task failure: a task fails when the agent books the wrong option, which looks correct to an auditor that does not know the customer's hidden goal, and solved tasks contain real process errors next to clear false alarms.

Manual review. The author reviewed 20 runs the 70B had flagged (10 failed, 10 solved tasks, drawn at random, outcomes hidden), asking one question per run: is the cited step really an error of the agent? 8 of 20 flags were real (40 %, 95 % interval 22 to 61 %): 5 of 10 on failed tasks, 3 of 10 on solved ones. In 7 of the 12 false alarms the reviewer noted a different real error in the same run. On an agent and domain it was not built for, the auditor often notices that a run went wrong but cites the wrong step or reason: localisation, not only detection, needs adaptation for a new domain (domain examples and policy wording tuned on a small dev set first).

6 Limitations

- **A proof of concept under a small budget.** One training run per configuration on 414 examples (section 3); production accuracy is not shown, and the two-tier recall gap (71 vs 101 broken runs flagged) is open.
- **Mostly synthetic data.** Our own simulator produced the training and test runs. On tau-bench the auditor does not predict task failure, and only 8 of 20 of its flags were real errors. Results on our domains do not carry over to a new domain without adaptation.
- **Recall in manufacturing.** The 8B screener misses errors the 70B would catch; the escalation rule cannot recover what the screener clears.
- **Two domains, one error per run.** Real runs can hold several errors and longer histories; coding agents and traces beyond 16k tokens were not tested (the 8B at 16k context failed on 41 of 100 tau-bench runs).
- **Test-set use and noise.** The 8B systems ran on the test set twice (before and after the harness fixes); the table uses the later run, both agree within about ±0.03. B0 and B3 ran with the earlier harness, which differs only in JSON repair. At 120 to 130 runs per set, differences below about 0.03 to 0.05 are not meaningful.
- **Confidence** is not calibrated (neither the finding confidence nor the GRPO probability), so cost cannot yet be traded against recall with a threshold. In the live demo the same run can take 4 s or loop for 90 s on the 8B; the token cap bounds the loop and escalates to the 70B.

7 Reproducibility

Commands for every step are in `docs/reproduce.md`; every number in this report maps to a folder in `results/README.md`. In short: `scripts/make_splits.sh` regenerates the data from the seeds above; `rosetta_flow.train.gold` and `train.sft` build the SFT set; `scripts/hf_train.sh` and `hf_rl.sh` train on Hugging Face Jobs; `scripts/hf_eval.sh` evaluates the 8B systems on vLLM, `rosetta_flow.harness` runs the 70B, `rosetta_flow.eval.cascade` computes the two tiers; `scripts/hf_offline.sh` is the offline proof. **Run the product:** set `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`, then `make run`; on-prem, point the tiers in `pipeline.toml` at local vLLM servers. **Tests:** 69 offline tests (`pytest`), including security tests (injection fence, ReDoS, fuzzing, tamper-evident review log).

8 From proof of concept to product

Three steps matter most, in this order. (1) **A pilot on real, consented traces** from a Swiss organisation, to measure on runs we did not simulate. (2) **Training compute**: thousands of 70B investigations instead of 414, several seeds and a search over settings, then preference training (DPO) on the 557 accepted vs 272 rejected investigations and RL on the investigation itself, aimed at the 8B's recall so fewer broken runs are cleared. (3) **An OpenTelemetry importer**, so any agent instrumented with `gen_ai.*` spans can be audited without a converter. Further extensions, with what each builds on, are in `docs/extensions.md`: new domains as kits (banking, insurance claims, public services), a confirmer that checks the screener's finding with tools (the tool-less version is built and measured, section 5.2), a thinking confirmer for hard cases, a guard that holds a high-risk tool call before it runs, retraining the screener on reviewers' decisions on premises, fleet analytics, and a quantised 8B on CPU for disconnected sites.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Swiss AI Initiative, *Apertus* (2025), https://arxiv.org/abs/2509.14233.
- Yao et al., *tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains* (2024), https://arxiv.org/abs/2406.12045.
- FINMA Guidance 08/2024 (governance and risk management for AI); EU AI Act (logging and human oversight for high-risk systems).
- vLLM, TRL (GRPO), PEFT; FHNW's text-only Apertus conversion `andreasmartin/apertus-v1.5-8b-text`.